

Optimización de Arquitecturas MLP

Javier Ramírez González

javier.ramirez@cicese.edu.mx

CICESE

Maestría en Ciencias de la Computación

Ensenada, Baja California, México

ABSTRACT

Este trabajo compara dos estrategias de optimización para la búsqueda automática de arquitecturas eficientes en perceptrones multicapa (MLP), un micro algoritmo genético (μ GA) con población de 4 individuos y reinicio elitista, y el algoritmo evolutivo L-SHADE con momentum, utilizando búsqueda aleatoria como línea base. La optimización se centra en las capas totalmente conectadas, empleando ResNet18 como extractor fijo de características para conjuntos de imágenes. Se evaluaron seis conjuntos de datos de diversos dominios (Iris [5], Heart Disease [6] y Credit Card Fraud Detection [1], mientras que para imágenes se consideran TinyFace [3] y CIFAR-10 [2]). Mediante 30 ejecuciones independientes sobre los conjuntos pequeños (Iris y Heart), se encontró que μ GA reduce significativamente el número de parámetros respecto a la búsqueda aleatoria y L-SHADE, alcanzando hasta 49.6% de reducción. En términos de precisión, ninguna estrategia superó significativamente a la búsqueda aleatoria. Se concluye que μ GA es significativamente mejor para encontrar arquitecturas más compactas, siendo preferible cuando se prioriza la reducción del tamaño del modelo al menos para esos dominios.

1 INTRODUCCIÓN

El aprendizaje profundo, en años recientes, se ha convertido en el estado del arte para tareas como la clasificación, debido a su capacidad para aprender representaciones jerárquicas directamente a partir de los datos, superando a los enfoques tradicionales basados en selección manual de características. Sin embargo, no existe una formulación matemática directa que permita determinar de manera óptima la configuración de una red neuronal, como el número de capas, neuronas, funciones de activación o conexiones. En la práctica, el diseño de estas arquitecturas suele realizarse mediante prueba y error, experiencia previa o siguiendo configuraciones reportadas en la literatura, lo cual implica un alto costo computacional.

Debido a este problema, la Búsqueda de Arquitecturas Neuronales (Neural Architecture Search, NAS) ha surgido como una alternativa para automatizar la exploración de configuraciones mediante algoritmos de optimización [4]. NAS puede definirse como el proceso de encontrar automáticamente una arquitectura neuronal óptima dentro de un espacio de búsqueda predefinido, guiado por una función objetivo que evalúa el desempeño del modelo.

No obstante, una de las principales limitaciones de NAS es su elevado costo computacional, ya que cada solución candidata requiere

ser evaluada, lo que generalmente implica entrenar (total o parcialmente) una red neuronal. En este trabajo se consideran dos enfoques: (i) el uso de perceptrones multicapa (MLP) para la optimización de arquitecturas en datos tabulares, y (ii) el uso de una red previamente entrenada, específicamente ResNet18, como extractor fijo de características en tareas de imágenes para un MLP. Este último enfoque permite trabajar con representaciones más compactas y enfocar la búsqueda únicamente en la capa clasificadora.

La motivación de este trabajo es encontrar un equilibrio adecuado entre precisión y complejidad del modelo. Aunque existe la tendencia de utilizar redes cada vez más grandes, esto incrementa el costo computacional y el riesgo de sobreajuste, mientras que arquitecturas demasiado pequeñas pueden generar subajuste. Por lo tanto, el objetivo no es encontrar la red más grande ni la más pequeña, sino una configuración eficiente que mantenga un buen desempeño de generalización.

Para abordar esta optimización, se comparan tres estrategias: un micro algoritmo genético (μ GA), L-SHADE y búsqueda aleatoria. El uso de micro algoritmos genéticos utiliza poblaciones pequeñas, reduciendo el número de evaluaciones necesarias, y mantienen la diversidad mediante mecanismos de reinicio elitista [7]. Por otro lado, L-SHADE es una variante de Evolución Diferencial que incorpora adaptación histórica de parámetros y reducción progresiva de la población, lo que la hace competitiva en problemas de optimización continua [8]. Finalmente, la búsqueda aleatoria se utiliza como línea base para determinar si las mejoras obtenidas por los métodos propuestos son resultado de una exploración guiada y no del azar.

Los métodos se evalúan en problemas de clasificación sobre datos tabulares e imágenes. Para datos tabulares se utilizan Iris [5], Heart Disease [6] y Credit Card Fraud Detection [1], mientras que para imágenes se consideran TinyFace [3] y CIFAR-10 [2]. El objetivo es maximizar el desempeño de clasificación y, al mismo tiempo, minimizar la complejidad del modelo, obteniendo arquitecturas más ligeras y eficientes.

1.1 Objetivo

El objetivo general de este trabajo es analizar el uso de algoritmos de optimización para reducir el número de parámetros de una arquitectura neuronal manteniendo una alta precisión. Específicamente, se busca obtener modelos más compactos sin perder capacidad de generalización, medida a través del desempeño en datos no vistos. El interés no radica en encontrar la red más pequeña ni la más precisa

de forma aislada, sino en estudiar el compromiso entre precisión y complejidad del modelo.

Adicionalmente, se espera que los métodos de optimización propuestos superen a la búsqueda aleatoria, demostrando que la obtención de buenas arquitecturas no es producto del azar, sino del uso de estrategias de exploración eficientes del espacio de búsqueda

2 CONCEPTOS PREVIOS

2.1 Aprendizaje por transferencia con ResNet18

En el ámbito del aprendizaje profundo aplicado a visión por computadora, entrenar una red convolucional desde cero requiere grandes conjuntos de datos y una capacidad de cómputo considerable. El *Transfer Learning* (aprendizaje por transferencia) reutiliza un modelo previamente entrenado en una tarea (típicamente ImageNet, con 1.2 millones de imágenes y 1000 clases) para resolver una tarea objetivo diferente.

En este trabajo se emplea **ResNet18** [?] como extractor fijo de características. La arquitectura ResNet18 está compuesta por 18 capas convolucionales organizadas en bloques residuales que mitigan el problema del desvanecimiento del gradiente. La red se carga con los pesos aprendidos en ImageNet y se modifica eliminando su última capa totalmente conectada (la capa de clasificación de 1000 neuronas). De este modo, la salida de la penúltima capa es un vector de **512 dimensiones** que constituye una representación semántica compacta de la imagen de entrada.

Formalmente, dado un conjunto de imágenes $\mathcal{I}$, se aplica la función de extracción $\Phi : \mathbb{R}^{224 \times 224 \times 3} \rightarrow \mathbb{R}^{512}$ definida por las capas convolucionales congeladas de ResNet18. Para cada imagen I_j se obtiene un vector de características:

$$\mathbf{z}_j = \Phi(I_j), \quad \mathbf{z}_j \in \mathbb{R}^{512}. \quad (1)$$

Los pesos de Φ no se actualizan durante el entrenamiento posterior, lo que reduce el costo computacional y permite utilizar lotes grandes de datos.

La utilidad de este enfoque radica en que las primeras capas de ResNet18 detectan bordes, texturas y formas genéricas, mientras que las capas intermedias capturan partes de objetos. Estas representaciones son suficientemente generales para transferirse a nuevos dominios visuales (como rostros en TinyFace o objetos en CIFAR-10) sin necesidad de reentrenar el extractor. El problema se reduce entonces a optimizar únicamente las capas totalmente conectadas (MLP) que procesan estos vectores de 512 dimensiones.

2.2 Arquitectura del Perceptrón Multicapa

El clasificador que se optimiza en este trabajo es un Perceptrón Multicapa (MLP) con exactamente **dos capas ocultas** ($m = 2$). La elección de esta profundidad fija no es arbitraria, sino que responde a criterios técnicos y metodológicos consolidados en la literatura de Búsqueda de Arquitectura Neuronal (NAS).

Estándar de evaluación en NAS. Existe un punto de referencia utilizado para la búsqueda conjunta de arquitectura e hiperparámetros que emplea un MLP con dos capas ocultas [7], lo que permite comparar la eficiencia de algoritmos de optimización (como μ GA o L-SHADE) sin requerir recursos computacionales masivos.

Equilibrio entre capacidad de representación y riesgo de sobreajuste. Desde el punto de vista teórico, un MLP con una única capa oculta es un aproximador universal, pero puede requerir un número exorbitante de neuronas para modelar fronteras de decisión complejas. Por otro lado, redes excesivamente profundas (tres o más capas) incrementan el número de parámetros y, con ello, el riesgo de sobreajuste cuando se trabaja con conjuntos de datos de tamaño moderado o con características altamente discriminativas como las extraídas por ResNet18.

Minimización de la complejidad estructural. El trabajo seminal de Landa et al. [7] adopta explícitamente una arquitectura de referencia con dos capas ocultas con el objetivo de *minimizar la complejidad del modelo*. La meta es hallar la red más compacta posible (en términos de neuronas por capa) que mantenga un rendimiento predictivo estadísticamente indistinguible del de una red densa completa.

Control de la explosión combinatoria. El espacio de búsqueda de arquitecturas neuronales crece de manera exponencial con cada capa adicional. Al fijar el número de capas ocultas en dos, el espacio de decisión se reduce a las dimensiones L_1 y L_2 (y, en su caso, la tasa de aprendizaje). Esta reducción es crítica para que metaheurísticas con poblaciones pequeñas (como el μ GA de 4 individuos) puedan explorar eficazmente el espacio y converger a soluciones de alta calidad dentro de presupuestos computacionales limitados.

2.3 Micro Algoritmo Genético (μ GA)

2.3.1 Dominio del problema. El μ GA opera en un dominio de optimización combinatoria discreta. Cada arquitectura candidata se representa mediante una cadena binaria de longitud fija. El espacio de búsqueda es

$$A \subset \{0, 1\}^B, \quad (2)$$

donde $B = 2 \cdot n_b$, siendo $n_b = \lceil \log_2(n_{\max} - n_{\min} + 1) \rceil$ el número de bits necesarios para codificar el número de neuronas de una capa oculta dentro del rango $[n_{\min}, n_{\max}]$. El tamaño del espacio de búsqueda es 2^B , que crece exponencialmente con el intervalo permitido.

La función objetivo $f : A \rightarrow \mathbb{R}$ carece de expresión analítica y actúa como una *caja negra*: para evaluar $f(\alpha)$ es necesario construir la arquitectura correspondiente, entrenar un MLP durante un número reducido de épocas y estimar su precisión balanceada mediante validación cruzada de 3 pliegues:

$$f(\alpha) = \frac{1}{3} \sum_{k=1}^3 \text{BalAcc}(\mathcal{M}_\alpha^{(k)}) - \lambda \cdot \mathbb{I}[\min_c \text{Recall}_c = 0], \quad (3)$$

donde $\lambda = 0.5$ penaliza a los modelos que ignoran por completo alguna clase.

2.3.2 Representación y decodificación. El vector de decisión $\alpha \in \{0, 1\}^B$ codifica el número de neuronas de las dos capas ocultas como $\alpha = (b_{1,1}, \dots, b_{1,n_b}, b_{2,1}, \dots, b_{2,n_b})$.

La decodificación a valores enteros se realiza mediante un mapeo lineal:

$$L_i = n_{\min} + \left\lfloor \frac{v_i}{2^{n_b} - 1} (n_{\max} - n_{\min}) \right\rfloor, \quad i = 1, 2, \quad (4)$$

donde $v_i = \sum_{j=1}^{n_b} b_{i,j} 2^{n_b-j}$ es el valor entero del bloque i .

2.3.3 Mecanismo de reinicio elitista. La población está compuesta por $N_p = 4$ individuos. Para contrarrestar la pérdida prematura de diversidad, se monitoriza la distancia de Hamming normalizada:

$$\bar{H}(\mathcal{P}) = \frac{2}{N_p(N_p - 1)} \sum_{i=1}^{N_p-1} \sum_{j=i+1}^{N_p} \frac{1}{B} \sum_{k=1}^B \mathbb{I}[p_{i,k} \neq p_{j,k}]. \quad (5)$$

Si $\bar{H}(\mathcal{P}) < \theta$ (con $\theta = 0.05$), se considera que la población ha convergido a una región homogénea. En ese caso, se preserva únicamente el individuo élite y los restantes se regeneran aleatoriamente.

Algorithm 1 Micro Algoritmo Genético (μ GA)

```

1: Entrada: datos  $X, y$ , límites  $n_{\min}, n_{\max}$ , tamaño de población
    $N_p = 4$ , generaciones máximas  $g_{\max}$ , probabilidad de cruce
    $p_c = 0.9$ , umbral de diversidad  $\theta = 0.05$ 
2: Calcular número de bits  $n_b = \lceil \log_2(n_{\max} - n_{\min} + 1) \rceil$ ,  $B \leftarrow 2 \cdot n_b$ 
3: Inicializar población  $P = \{c_1, \dots, c_{N_p}\}$  con  $c_i \in \{0, 1\}^B$  aleatorios
4: for cada  $c_i \in P$  do
5:   Decodificar  $c_i \rightarrow \alpha_i = (L_1, L_2)$ 
6:   Entrenar MLP con arquitectura  $\alpha_i$  (3-pliegues, pocas épocas)
7:   Evaluar  $f_i = f(\alpha_i)$ 
8: end for
9:  $g \leftarrow 0$ 
10: while  $g < g_{\max}$  do
11:   Seleccionar padres mediante torneo binario (empate: menor  $L_1 + L_2$ )
12:   Aplicar cruce de dos puntos con probabilidad  $p_c$  para obtener descendencia  $O$ 
13:   Evaluar cada individuo de  $O$  y actualizar sus aptitudes
14:   Formar nueva población  $P$  con los  $N_p$  mejores de  $P \cup O$ 
15:   if aptitud del mejor individuo actual  $>$  mejor histórica then
16:     Actualizar mejor arquitectura  $\alpha^*$  y su aptitud  $f^*$ 
17:   end if
18:   Calcular diversidad  $d_H = \bar{H}(P)$   $\triangleright$  Distancia de Hamming normalizada
19:   if  $d_H < \theta$  then
20:     Preservar el mejor individuo de  $P$  (élite)
21:     Generar aleatoriamente los otros  $N_p - 1$  individuos
22:     Re-evaluar los nuevos individuos
23:   end if
24:    $g \leftarrow g + 1$ 
25: end while
26: Salida: mejor arquitectura  $\alpha^*$  y su aptitud  $f(\alpha^*)$ 

```

2.4 L-SHADE con Momentum

2.4.1 Dominio del problema. L-SHADE es un algoritmo de Evolución Diferencial (DE) que opera sobre un dominio continuo. Cada solución candidata se representa como un vector en el hipercubo unitario. El espacio de búsqueda es

$$A \subset [0, 1]^D, \quad (6)$$

donde $D = 2$ corresponde a las dos capas ocultas. Aunque la decodificación redondea los valores a enteros, la búsqueda continua permite aprovechar los operadores de DE y la adaptación fina de parámetros.

La función objetivo $f : A \rightarrow \mathbb{R}$ es la misma que en el μ GA: costosa de evaluar y sin expresión analítica.

2.4.2 Representación y decodificación. Un vector $\mathbf{x} \in [0, 1]^2$ se decodifica como:

$$L_i = \text{round}(n_{\min} + x_i \cdot (n_{\max} - n_{\min})), \quad i = 1, 2, \quad (7)$$

con saturación para garantizar $L_i \in [n_{\min}, n_{\max}]$.

2.4.3 Adaptación de parámetros y LPSR. El tamaño de población se reduce linealmente con el número de evaluaciones (NFE):

$$NP_t = \text{round} \left(NP_{\text{init}} + (NP_{\min} - NP_{\text{init}}) \cdot \frac{\text{NFE}}{\text{NFE}_{\max}} \right). \quad (8)$$

Los factores de escala F y tasas de cruce CR se muestrean de memorias históricas $\mathbf{M}_F, \mathbf{M}_{CR}$ de tamaño $H = 5$. En cada generación se selecciona un índice aleatorio $r \in \{0, \dots, H - 1\}$ y se generan:

$$F \sim \text{Cauchy}(\mathbf{M}_F[r], 0.1), \quad CR \sim \mathcal{N}(\mathbf{M}_{CR}[r], 0.1), \quad (9)$$

ambos truncados a $[0, 1]$.

2.4.4 Mutación con momentum. Se emplea la estrategia *current-to-pbest/1* con un término adicional de momentum:

$$\mathbf{v}_i = \mathbf{x}_i + F \cdot (\mathbf{x}_{\text{pbest}} - \mathbf{x}_i) + F \cdot (\mathbf{x}_{r1} - \mathbf{x}_{r2}) + \gamma \cdot \mathbf{m}_i, \quad (10)$$

donde $\mathbf{x}_{\text{pbest}}$ se elige aleatoriamente entre el $p = 10\%$ superior de la población, $\mathbf{x}_{r1}$ y $\mathbf{x}_{r2}$ son individuos distintos, y $\mathbf{m}_i$ es la velocidad acumulada del individuo i . El coeficiente $\gamma = 0.5$ controla la inercia.

2.4.5 Actualización de la velocidad y memoria histórica. Si el vector de prueba $\mathbf{u}_i$ mejora al padre $\mathbf{x}_i$, se actualiza la velocidad:

$$\mathbf{m}_i \leftarrow \mathbf{u}_i - \mathbf{x}_i, \quad (11)$$

y los valores F, CR exitosos se almacenan en los conjuntos S_F, S_{CR} . Al final de la generación se actualizan las memorias con la media ponderada de Lehmer:

$$\mathbf{M}_F[k] = \frac{\sum_{w \in S_F} w^2}{\sum_{w \in S_F} w}, \quad \mathbf{M}_{CR}[k] = \frac{1}{|S_{CR}|} \sum_{c \in S_{CR}} c, \quad (12)$$

donde los pesos w son proporcionales a la magnitud de la mejora obtenida.

2.4.6 Reinicio suave. Si la diversidad poblacional, medida como la desviación estándar promedio de las coordenadas, cae por debajo de un umbral ($\theta = 0.05$), se preserva el mejor individuo y el resto se regenera aleatoriamente en $[0, 1]^D$, reiniciando también sus velocidades.

Algorithm 2 L-SHADE con Momentum

```

1: Entrada:  $X, y, n_{\min}, n_{\max}, NP_{init}, NP_{min}, NFE_{max}, H, p, \gamma, \theta$ 
2: Inicializar  $P \subset [0, 1]^D, M_F, M_{CR}, V = 0$ 
3: Evaluar  $f(x_i)$  para todo  $x_i \in P$ 
4:  $NFE \leftarrow NP_{init}$ , guardar mejor  $\alpha^*$ 
5: while  $NFE < NFE_{max}$  do
6:   Ajustar  $NP$  y seleccionar mejores individuos
7:    $S_F, S_{CR} \leftarrow \emptyset$ 
8:   for  $i = 1$  to  $NP$  do
9:     Muestrear  $F, CR$ ; seleccionar  $x_{pbest}, r1, r2$ 
10:     $v \leftarrow x_i + F(x_{pbest} - x_i) + F(x_{r1} - x_{r2}) + \gamma V[i]$ 
11:    Cruce binomial  $\rightarrow u$ ; evaluar  $f(u)$ ;  $NFE++$ 
12:    if  $f(u) \geq f_i$  then
13:       $x_i \leftarrow u$ , actualizar  $V[i]$ 
14:      Guardar  $F, CR$ ; actualizar mejor  $\alpha^*$ 
15:    end if
16:  end for
17:  if  $S_F \neq \emptyset$  then
18:    Actualizar  $M_F, M_{CR}$ 
19:  end if
20:  if Diversidad( $P$ )  $< \theta$  then
21:    Reinicio elitista
22:  end if
23: end while
24: Salida:  $\alpha^*$ 

```

3 METODOLOGÍA

3.1 Adquisición y procesamiento de datos

Se emplearon seis conjuntos de datos de dominios diversos, divididos en dos categorías: tabulares e imágenes. La Tabla 1 resume sus características principales.

Table 1: Conjuntos de datos utilizados.

Dataset	Tipo	Muestras	Clases
Iris	Tabular	150	3
Heart Disease	Tabular	303	2
Credit Card Fraud	Tabular	284,807	2
Mammographic	Tabular	961	2
CIFAR-10	Imagen	60,000	10
TinyFace	Imagen	~3M	2,570

En algunos experimentos se utilizó únicamente una porción de los datos originales para reducir el costo computacional.

Datos tabulares. Se cargaron desde archivos CSV o mediante scikit-learn. Las variables numéricas se normalizaron con StandardScaler (media cero, varianza unitaria). Se realizó una partición estratificada 80/20 para entrenamiento y prueba. En casos donde la estratificación fallaba por clases con una sola muestra, se utilizó partición aleatoria simple.

Datos de imagen. Las imágenes se redimensionaron a 224×224 píxeles y se normalizaron con los estadísticos de ImageNet (media $[0.485, 0.456, 0.406]$, desviación $[0.229, 0.224, 0.225]$). Para TinyFace,

se limitó el número de identidades a 200 para mantener un tamaño manejable.

3.2 Extracción de características con ResNet18

Para los conjuntos de imágenes (CIFAR-10 y TinyFace), se empleó Transfer Learning utilizando ResNet18 previamente entrenada en ImageNet, se utiliza como extractor fijo de características.

- (1) Se cargó el modelo ResNet18 con pesos de ImageNet.
- (2) Se eliminó la última capa totalmente conectada (clasificación de 1000 clases).
- (3) Se congelaron todos los pesos de las capas convolucionales.
- (4) Para cada imagen, se obtuvo un vector de 512 dimensiones correspondiente a la salida de la penúltima capa.
- (5) Los vectores extraídos se normalizaron.

Este proceso transforma el problema de clasificación de imágenes en un problema de clasificación tabular sobre vectores de 512 características, reduciendo drásticamente el costo computacional de las evaluaciones posteriores.

3.3 Espacio de búsqueda y función objetivo

Arquitectura del MLP. Se optimizó un Perceptrón Multicapa con exactamente dos capas ocultas. Cada arquitectura candidata queda definida por el par (L_1, L_2) , donde $L_i \in [n_{\min}, n_{\max}]$ es el número de neuronas de la capa i . Los límites $n_{\min}$ y $n_{\max}$ se ajustaron por dataset.

Función objetivo. Para evaluar una arquitectura candidata $\alpha = (L_1, L_2)$, se utilizó validación cruzada estratificada de 3 pliegues sobre el conjunto de entrenamiento. En cada pliegue, el MLP se entrenó durante un número reducido de épocas (entre 5 y 12, según el dataset) usando el optimizador Adam con tasa de aprendizaje 10^{-3} y *batch size* de 32. La métrica de desempeño fue la precisión balanceada:

$$\text{BalAcc} = \frac{1}{C} \sum_{c=1}^C \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}. \quad (13)$$

La aptitud final se calculó como:

$$f(\alpha) = \frac{1}{3} \sum_{k=1}^3 \text{BalAcc}_k - 0.5 \cdot \mathbb{I}[\min_c \text{Recall}_c = 0], \quad (14)$$

donde el término de penalización se activa únicamente para datasets con desbalance extremo (ratio > 10), desalentando arquitecturas que ignoren completamente alguna clase minoritaria.

Mecanismo de caché. Para evitar re-evaluar arquitecturas idénticas, se implementó un sistema de caché basado en hashes MD5. Cada evaluación se almacena en un archivo CSV, permitiendo recuperar la aptitud de arquitecturas previamente evaluadas.

3.4 Algoritmos de optimización

Se implementaron y compararon tres estrategias de optimización:

Micro Algoritmo Genético (μ GA). Opera sobre una población de 4 individuos codificados en binario. Cada capa oculta se representa con $n_b = \lceil \log_2(n_{\max} - n_{\min} + 1) \rceil$ bits. La decodificación se realiza mediante mapeo lineal. En cada generación se aplica selección por torneo binario (con desempate por menor suma de neuronas), cruce de dos puntos con probabilidad $p_c = 0.9$, y elitismo. Cuando la diversidad poblacional (distancia de Hamming normalizada) cae por debajo de 0.05, se preserva el elite y los otros tres individuos se regeneran aleatoriamente. El algoritmo ejecuta $g_{\max}$ generaciones (entre 100 y 150 según el dataset).

L-SHADE con Momentum. Opera sobre un espacio continuo $[0, 1]^2$. La población inicial es de 20 individuos y se reduce linealmente hasta un mínimo de 4 con el número de evaluaciones (LPSR). Los parámetros F (factor de escala) y CR (tasa de cruce) se adaptan mediante memoria histórica de tamaño $H = 5$. La mutación emplea la estrategia *current-to-pbest/1* con un término adicional de momentum:

$$\mathbf{v}_i = \mathbf{x}_i + F(\mathbf{x}_{pbest} - \mathbf{x}_i) + F(\mathbf{x}_{r1} - \mathbf{x}_{r2}) + 0.5 \cdot \mathbf{m}_i. \quad (15)$$

La velocidad $\mathbf{m}_i$ se actualiza como $\mathbf{u}_i - \mathbf{x}_i$ cuando el vector de prueba $\mathbf{u}_i$ mejora al padre. Si la diversidad (desviación estándar promedio) cae por debajo de 0.05, se aplica un reinicio suave preservando el elite. El algoritmo se ejecuta hasta alcanzar $NFE_{\max}$ evaluaciones (entre 1000 y 5000 según el dataset).

Búsqueda Aleatoria (Random Search). Muestra arquitecturas uniformemente en $[n_{\min}, n_{\max}]^2$ durante un número fijo de intentos ($NFE_{\max}/2$). Sirve como línea base para validar la efectividad de los métodos guiados.

3.5 Entrenamiento final y evaluación

Para cada dataset y algoritmo, una vez encontrada la mejor arquitectura α^* , se realizó un entrenamiento completo sobre el conjunto de entrenamiento (80% de los datos).

Se evaluó el modelo resultante sobre el conjunto de prueba (20% de los datos), registrando:

- *Accuracy* estándar y balanceada.
- Número de parámetros entrenables.
- Porcentaje de reducción de parámetros respecto a la arquitectura base.
- SDD (*Standard Deviation of Differences*) como indicador de sobreajuste:

$$SDD = \sigma(\text{Acc}_{\text{train}} - \text{Acc}_{\text{val}}, \text{Acc}_{\text{val}} - \text{Acc}_{\text{test}}, \text{Acc}_{\text{train}} - \text{Acc}_{\text{test}}). \quad (16)$$

4 RESULTADOS

Se realizaron 30 ejecuciones independientes de cada algoritmo (μ GA, L-SHADE y Random Search) sobre los conjuntos de datos Iris y Heart Disease. Para cada ejecución se registró la arquitectura optimizada encontrada, el número de parámetros entrenables, la precisión (accuracy) sobre el conjunto de prueba y el porcentaje de reducción de parámetros respecto a la arquitectura base. La Tabla 2 resume los resultados obtenidos.

Table 2: Resultados promedio de 30 ejecuciones independientes sobre Iris y Heart Disease, comparando contra la arquitectura base.

Dataset	Alg.	Parámetros	Acc.	Red.	p (par)	p (acc)
3*Iris (base: 4931)	RAND	3204 $\pm$ 793	0.841 $\pm$ 0.044	35.0%	< 0.0001*	0.2843
	μ GA	2804 $\pm$ 839	0.828 $\pm$ 0.051	43.1%	< 0.0001*	0.0265*
	LSH	3386 $\pm$ 771	0.831 $\pm$ 0.052	31.3%	< 0.0001*	0.0259*
3*Heart (base: 5442)	RAND	3646 $\pm$ 939	0.844 $\pm$ 0.029	33.0%	< 0.0001*	0.3725
	μ GA	2740 $\pm$ 754	0.828 $\pm$ 0.035	49.6%	< 0.0001*	0.3308
	LSH	3382 $\pm$ 936	0.848 $\pm$ 0.031	37.9%	< 0.0001*	0.1249

* Diferencia estadísticamente significativa ($p < 0.05$).

4.1 Análisis de significancia estadística

Los conjuntos Iris y Heart Disease, los cuales fueron seleccionados por su tamaño reducido que permite múltiples repeticiones en tiempo razonable.

Reducción de parámetros. Los tres algoritmos logran una disminución estadísticamente significativa en el número de parámetros entrenables respecto a la arquitectura base ($p < 0.0001$ en todos los casos). μ GA destaca con la mayor reducción promedio: 43.1% en Iris y 49.6% en Heart, evidenciando su capacidad para identificar arquitecturas notablemente más compactas.

Precisión de clasificación. En el conjunto Iris, tanto μ GA como L-SHADE exhiben una ligera caída en la precisión que resulta estadísticamente significativa ($p = 0.0265$ y $p = 0.0259$, respectivamente), mientras que Random Search no muestra diferencia significativa ($p = 0.2843$). La magnitud de esta disminución es reducida (aproximadamente 3 puntos porcentuales), por lo que podría considerarse aceptable en contextos donde se priorice la eficiencia del modelo. En Heart Disease, ninguno de los tres algoritmos presenta una diferencia estadísticamente significativa en precisión ($p > 0.05$ en todos los casos). Esto indica que, para este problema, las arquitecturas reducidas mantienen un poder predictivo comparable al modelo completo, confirmando que una porción considerable de los parámetros originales es redundante.

Limitaciones del análisis estadístico. Es importante señalar que el análisis con 30 ejecuciones independientes se restringió solamente a los conjuntos Iris y Heart Disease debido a su bajo costo computacional. Los conjuntos restantes (TinyFace, CIFAR-10, Credit Card Fraud y Mammographic Masses) poseen un número de muestras y una dimensionalidad mayores, lo que eleva el tiempo de evaluación de cada arquitectura candidata a varios minutos u horas. Realizar 30 repeticiones completas de cada algoritmo sobre estos datos requeriría semanas de cómputo continuo, un recurso fuera del alcance de este trabajo. Los hallazgos en los conjuntos pequeños proporcionan evidencia inicial de la efectividad de los métodos propuestos, aunque su generalización a problemas de mayor escala debe de probarse y se plantea como trabajo futuro sujeto a disponibilidad de recursos computacionales.

4.2 Compromiso entre precisión y complejidad

La Figura 1 ilustra el compromiso (*trade-off*) entre la precisión de clasificación y el número de parámetros para cada algoritmo en ambos conjuntos de datos. Cada punto representa una ejecución independiente, y las elipses muestran la dispersión de cada método.

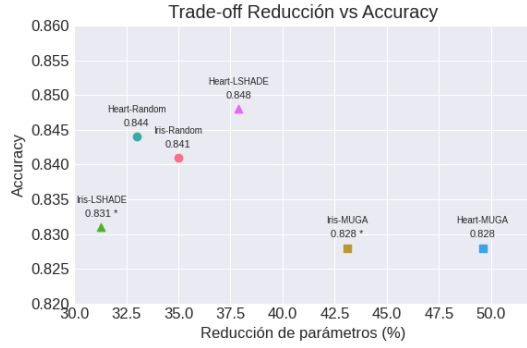


Figure 1: Compromiso entre precisión (accuracy) y complejidad (número de parámetros) para los tres algoritmos evaluados sobre Iris y Heart Disease. Las elipses representan la dispersión de las 30 ejecuciones.

- μ GA explora consistentemente arquitecturas con menor número de parámetros, aunque con una varianza ligeramente mayor en precisión.
- L-SHADE tiende a posicionarse en una región intermedia, balanceando reducción de parámetros con preservación de la precisión.
- Random Search muestra una dispersión más amplia, reflejando la naturaleza no guiada de su exploración.

4.3 Resultados de ejecuciones individuales sobre todos los conjuntos de datos

Además del análisis estadístico con 30 repeticiones para Iris y Heart, se realizó una ejecución individual de cada algoritmo sobre la totalidad de los seis conjuntos de datos considerados. La Tabla 3 presenta la precisión en prueba (*Test*), el porcentaje de reducción de parámetros y el indicador de sobreajuste SDD para la arquitectura base y la optimizada. Dado que se trata de una única ejecución por caso, estos resultados deben interpretarse con cuidado como una exploración inicial del comportamiento de los algoritmos, y no como evidencia concluyente de su desempeño relativo.

Table 3: Resultados de una ejecución individual por algoritmo y dataset.

Dataset	Alg.	Acc. Base	Acc. Opt.	Reducción	SDD Base/Opt.
TinyFace	μ GA	0.3924	0.3924	4.1%	0.2514 / 0.2499
	L-SHADE	0.3734	0.4241	1.9%	0.2685 / 0.2335
	Random	0.4114	0.3671	12.5%	0.2417 / 0.2417
CIFAR-10	μ GA	0.8973	0.8959	13.5%	0.0274 / 0.0224
	L-SHADE	0.8953	0.8964	+6.0%	0.0282 / 0.0286
	Random	0.8980	0.8949	8.7%	0.0273 / 0.0265
Iris	μ GA	0.3333	0.3333	31.9%	0.0118 / 0.0236
	L-SHADE	0.4333	0.6333	31.9%	0.0297 / 0.1021
	Random	0.9333	0.3333	31.9%	0.0453 / 0.0118
Heart	μ GA	0.7000	0.4833	8.8%	0.0117 / 0.0351
	L-SHADE	0.5833	0.5500	8.8%	0.0068 / 0.0181
	Random	0.8500	0.4667	8.8%	0.0904 / 0.0048
Credit	μ GA	0.9952	0.9982	18.0%	0.0014 / 0.0000
	L-SHADE	0.9982	0.9982	18.0%	0.0000 / 0.0000
	Random	0.9992	0.9982	18.0%	0.0002 / 0.0000
Mammo	μ GA	0.7590	0.7651	30.9%	0.0199 / 0.0611
	L-SHADE	0.7771	0.7470	30.9%	0.0213 / 0.0492
	Random	0.7470	0.7952	30.9%	0.0085 / 0.0295

Se reporta la precisión sobre el conjunto de prueba, la reducción de parámetros respecto a la arquitectura base y el SDD.

- En conjuntos de imágenes de alta complejidad como TinyFace, L-SHADE logró una mejora notable en precisión con una reducción mínima de parámetros, mientras que μ GA mantuvo la precisión original. Random Search, por el contrario, experimentó una caída en el rendimiento.
- En CIFAR-10, todos los algoritmos conservaron precisiones muy similares a la base, con reducciones de parámetros de hasta el 13.5%. El caso atípico de L-SHADE indica que la arquitectura encontrada incrementó ligeramente el número de parámetros.
- Para los conjuntos tabulares pequeños (Iris y Heart), los resultados de esta ejecución única difieren marcadamente del comportamiento promedio observado en las 30 repeticiones, por lo que es importante realizar múltiples ejecuciones para obtener conclusiones fiables.
- En Credit Card Fraud, todos los métodos preservaron la precisión casi perfecta del modelo base con una reducción uniforme del 18%, pero sin encontrar una arquitectura de dos capas que sea aceptable en la clase más pequeña.
- En Mammographic Masses, se observan variaciones en la precisión entre algoritmos, aunque todos logran una reducción de parámetros del 30.9%.

En conjunto, estas observaciones preliminares refuerzan la necesidad de realizar un número suficiente de ejecuciones independientes para poder emitir juicios estadísticamente fundamentados. Los resultados de las 30 repeticiones en Iris y Heart proporcionan una base, mientras que los datos de una sola corrida en los demás conjuntos sirven como punto de partida.

5 CONCLUSIONES Y REFLEXIONES

5.1 Conclusiones generales

Los resultados obtenidos permiten matizar la expectativa inicial de que los algoritmos evolutivos superarían claramente a la búsqueda aleatoria.

- **En reducción de parámetros, μ GA sí es mejor.** Consistentemente encuentra arquitecturas más compactas que Random Search y L-SHADE, alcanzando casi un 50% de reducción en Heart. Esto valida el diseño del algoritmo: población pequeña con reinicio elitista explora eficientemente el espacio en busca de simplicidad en esos dominios.
- **En precisión, ningún algoritmo es claramente superior.** En Heart, los tres métodos son estadísticamente equivalentes al modelo base. En Iris, μ GA y L-SHADE empeoran ligeramente la precisión, mientras que Random Search la mantiene. Esto sugiere que, para espacios de búsqueda reducidos como los evaluados, muestrear aleatoriamente es una estrategia aceptable si únicamente interesa buena precisión.
- **L-SHADE ocupa un punto intermedio.** No iguala la agresividad de μ GA en la reducción de parámetros ni la consistencia de Random Search en preservar la precisión. Su mecanismo adaptativo, diseñado para problemas continuos de alta dimensionalidad, no muestra una ventaja decisiva frente a la búsqueda aleatoria.
- El uso de Transfer Learning con ResNet18 como extractor fijo de características es una estrategia para reducir la dimensionalidad del problema en conjuntos de imágenes, permitiendo que la optimización se concentre exclusivamente en las capas clasificadoras. Esta reducción del espacio de búsqueda es necesaria para que algoritmos con poblaciones pequeñas como el μ GA puedan converger a soluciones de calidad en tiempos razonables.
- El mecanismo de reinicio elitista basado en distancia de Hamming implementado en μ GA demostró su utilidad para mantener la diversidad genética a pesar de contar con solo 4 individuos, permitiendo explorar regiones del espacio de búsqueda que de otro modo serían inaccesibles.
- La principal limitación identificada es la imposibilidad de realizar un análisis estadístico completo sobre todos los conjuntos de datos debido al alto costo computacional.

5.2 Reflexión

El hecho de que Random Search compite en precisión a métodos guiados no es trivial. Es debido a que el espacio de arquitecturas de dos capas ocultas, acotado a los rangos evaluados, contiene múltiples configuraciones aceptables, y que encontrarlas no requiere necesariamente mecanismos evolutivos sofisticados. Esto es particularmente cierto para conjuntos pequeños como Iris y Heart, donde el costo de evaluar una mala arquitectura es bajo y la varianza entre ejecuciones domina las diferencias entre algoritmos.

Sin embargo, la ventaja de μ GA en reducción de parámetros es importante para obtener modelos con la mitad de parámetros entrenables sin pérdida significativa de precisión (en Heart) o con pérdida mínima (en Iris) justifica su uso cuando el objetivo principal es la eficiencia computacional en inferencia, que es precisamente el objetivo de NAS.

La principal limitación de este estudio es que el análisis estadístico solo fue viable para los conjuntos pequeños. En problemas de mayor escala (TinyFace, CIFAR-10), donde cada evaluación es costosa, no se pudo establecer si las tendencias observadas se mantienen.

REFERENCES

- [1] [n.d.]. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- [2] [n.d.]. https://www-cs-toronto-edu.translate.google/~kriz/cifar.html?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc
- [3] Mohammad Subhi Al-Batah, Mowafaq Alzboon, and Muhyeddin Alqaraleh. 2024. Optimizing Genetic Algorithms with Multilayer Perceptron Networks for Enhancing TinyFace Recognition. *Data Metadata* 3 (Dec. 2024). <https://doi.org/10.56294/dm2024.594>
- [4] Sasan Salmani Pour Avval, Nathan D. Eskue, Roger M. Groves, and Vahid Yaghoubi. 2025. Systematic review on neural architecture search. *Artificial Intelligence Review* 58, 3 (Jan. 2025). <https://doi.org/10.1007/s10462-024-11058-w>
- [5] R. A. Fisher. 1936. Iris. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C56C76>.
- [6] Steinbrunn William Pfisterer Matthias Janosi, Andras and Robert Detrano. 1989. Heart Disease. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C52P4X>.
- [7] Ricardo Landa, David Tovas-Alanis, and Gregorio Toscano. 2024. Optimization of Deep Neural Networks Using a Micro Genetic Algorithm. *AI* 5 (2024), 2651–2679. <https://doi.org/10.3390/ai5040127>
- [8] Kai Zhang, Songhua Hu, Lianghong Zhu, Yu Han, and Qiujuan Huang. 2024. L-SHADE-MA: Enhancing L-SHADE with Momentum and Adaptive Decay Coefficient. (2024). <https://doi.org/10.21203/rs.3.rs-4160171/v1>